

Web Speech API

У цьому tutorialі ми створимо 4 невеликих додатки, в яких використовуються майже всі можливості, що надаються [Web Speech API](#) - по 2 додатки на кожен з інтерфейсів, що входять до складу [Web Speech API](#): [SpeechRecognition](#) та [SpeechSynthesis](#).

Додаток для озвучування тексту (перетворення тексту в мову)

Наша розмітка буде виглядати наступним чином:

```
<div id="wrapper">
  <h1>Speech Synthesis - Player</h1>
  <label>
    >Text:
    <textarea id="textarea">Привіт! Як справи?</textarea>
  </label>
  <label>
    >Voice:
    <select id="select"></select>
  </label>
  <label>
    >Volume:
    <input id="volume" type="range" min="0" max="1" step="0.1" value="1" />
    <span>1</span>
  </label>
  <label>
    >Rate:
    <input id="rate" type="range" min="0" max="3" step="0.5" value="1" />
    <span>1</span>
  </label>
  <label>
    >Pitch:
    <input id="pitch" type="range" min="0" max="2" step="0.5" value="1" />
    <span>1</span>
  </label>
  <div id="buttons">
    <button class="speak">Speak</button>
    <button class="cancel">Cancel</button>
    <button class="pause">Pause</button>
    <button class="resume">Resume</button>
  </div>
</div>
```

У нас є поле для введення тексту ([textarea](#)), який ми будемо озвучувати; випадючий список ([select](#)) для вибору голосу, яким буде озвучуватися текст; інпути-діапазони для встановлення гучності ([volume](#)), швидкості відтворення ([rate](#)) та висоти голосу ([pitch](#)), а також кнопки для запуску

(*speak*), скасування (*cancel*), призупинення (*pause*) та продовження (*resume*) відтворення. Загалом, нічого особливого.

Зверніть увагу на атрибути інпутів *min*, *max*, *step* та *value*. Значення цих атрибутів взяті з чернетки специфікації, але деякі з них віддані на відкуп виробникам браузерів, тобто залежать від конкретної реалізації. Також зверніть увагу на наявність майже у всіх елементів ідентифікаторів. Ми будемо використовувати ці *id* для прямого доступу до елементів у скрипті з метою скорочення коду, однак у реальних додатках так краще не робити, щоб уникнути забруднення глобального простору імен.

Переходимо до *JavaScript*. Створюємо екземпляр *SpeechSynthesisUtterance* ("utterance" можна перекласти як "вираження тексту словами"):

```
const U = new SpeechSynthesisUtterance()
```

Намагаємося отримати доступні голоси (саме "намагаємося", оскільки в перший раз, принаймні, в *Chrome* повертається порожній масив):

```
let voices = speechSynthesis.getVoices()
```

При виклику методу *getVoices()* виникає подія *voiceschanged*. Обробляємо цю подію для "справжнього" отримання голосів та формування випадального списку:

```
speechSynthesis.onvoiceschanged = () => {  
  voices = speechSynthesis.getVoices()  
  populateVoices(voices)  
}
```

Об'єкт голоса (назовем его так) выглядит следующим образом:

```
0: SpeechSynthesisVoice  
  default: true  
  lang: "de-DE"  
  localService: false  
  name: "Google Deutsch"  
  voiceURI: "Google Deutsch"
```

Реалізуем функцию формирования выпадающего списка:

```
function populateVoices(voices) {  
  // Перебираем голоса и создаем элемент `option` для каждого  
  // Текстовым содержимым `option` является название голоса, а значением -  
  индекс голоса в массиве голосов
```

```

voices.forEach((voice, index) => {
  select.options[index] = new Option(voice.name, index)
})

// Делаем голосом по умолчанию `Google русский`
// Он нравится мне больше, чем голос от `Microsoft`
const defaultVoiceIndex = voices.findIndex(
  (voice) => voice.name === 'Google русский'
)
select.selectedIndex = defaultVoiceIndex
// После этого инициализируем обработчики событий
initializeHandlers()
}

```

Функция инициализации обработчиков событий:

```

function initializeHandlers() {
  // Ниже перечислены почти все события, которые возникают при работе с
  // рассматриваемым интерфейсом
  U.onstart = () => console.log('Started')
  U.onend = () => console.log('Finished')
  U.onerror = (err) => console.error(err)
  // Мне не удалось добиться возникновения этих событий
  U.onpause = () => console.log('Paused')
  U.onresume = () => console.log('Resumed')

  // Обработка изменения настроек
  wrapper.onChange = ({ target }) => {
    if (target.type !== 'range') return
    handleChange(target)
  }

  // Обработка нажатия кнопок
  buttons.addEventListener('click', ({ target: { className } }) => {
    // SpeechSynthesis предоставляет такие методы как `speak()`, `cancel()`,
    // `pause()` и `resume()`
    // Вызов метода `speak()` требует предварительной подготовки
    // Кроме того, мы проверяем наличие текста в очереди на озвучивание с
    // помощью атрибута `speaking`
    // Есть еще два атрибута: `pending` и `paused`, но они не всегда возвращают
    // корректные значения
    switch (className) {
      case 'speak':
        if (!speechSynthesis.speaking) {
          convertTextToSpeech()
        }
        break
      case 'cancel':
        return speechSynthesis.cancel()
      case 'pause':
        return speechSynthesis.pause()
    }
  })
}

```

```

        case 'resume':
            return speechSynthesis.resume()
        default:
            return
    }
})
}

```

Обробка зміни налаштувань:

```

function handleChange(e1) {
    e1.nextElementSibling.textContent = e1.value
}

```

Функція перетворення тексту в мову:

```

function convertTextToSpeech() {
    // Отримуємо текст
    const trimmed = textarea.value.trim()
    if (!trimmed) return
    // Передаємо його екземпляру `SpeechSynthesisUtterance`
    U.text = trimmed
    // Отримуємо вибраний голос
    const voice = voices[select.value]
    // Передаємо голос та інші налаштування екземпляру
    U.voice = voice
    // мова
    U.lang = voice.lang
    // гучність
    U.volume = volume.value
    // швидкість
    U.rate = rate.value
    // висота голосу
    U.pitch = pitch.value
    // Запускаємо озвучування!
    speechSynthesis.speak(U)
}

```

Після встановлення всіх налаштувань наш екземпляр `SpeechSynthesisUtterance` виглядає наступним чином:

```

SpeechSynthesisUtterance
  lang: "uk-UA"
  onboundary: null
  onend: () => console.log('Finished')
  onerror: (err) => console.error(err)

```

```
onmark: null
onpause: () => console.log('Paused')
onresume: () => console.log('Resumed')
onstart: () => console.log('Started')
pitch: 1
rate: 1
text: "Привіт! Як справи?"
voice: SpeechSynthesisVoice { voiceURI: "Google український", name: "Google український", lang: "uk-UA", localService: false, default: false }
volume: 1
```

Додаємо можливість керування відтворенням за допомогою клавіатури:

```
window.onkeydown = ({ key }) => {
  switch (key.toLowerCase()) {
    case 's':
      if (!speechSynthesis.speaking) {
        convertTextToSpeech()
      }
      break
    case 'c':
      return speechSynthesis.cancel()
    case 'p':
      return speechSynthesis.pause()
    case 'r':
      return speechSynthesis.resume()
    default:
      return
  }
}
```

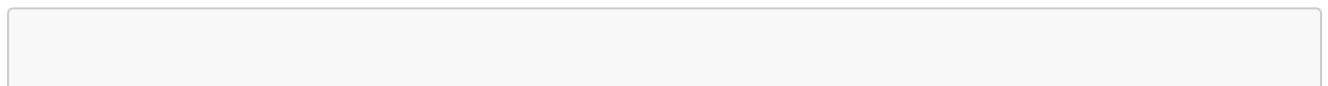
Пограти з кодом можна [тут](#).

Сторінка з можливістю озвучування текстового вмісту

Що стосується **SpeechSynthesis**, то з точки зору практичного використання цього інтерфейсу, складно щось придумати, окрім створення сторінок з можливістю озвучування розміщеного на них тексту. Цим ми і займемося.

Перевага використання **SpeechSynthesis** полягає в реальному покращенні доступності контенту додатку для людей з обмеженими можливостями здоров'я. Основна проблема, на мій погляд, полягає в тому, що користувачі не звикли до подібних додатків, отже, їм потрібно якимось чином натякнути про існування такої можливості. Причому, зробити це потрібно в дуже зрозумілій, але при цьому ненав'язливій формі.

Наша розмітка буде виглядати так:



```

<div id='wrapper'>
  <h1>Speech Synthesis - Page Reader</h1>
  <div>
    <button class='play' tabindex='1'></button>
    <p>
      JavaScript — мультипарадигмова мова програмування. Підтримує
      об'єктно-орієнтований, імперативний та функціональний стилі. Є
      реалізацією специфікації ECMAScript (стандарт ECMA-262).
    </p>
  </div>
  <div>
    <button class='play' tabindex='2'></button>
    <p>
      JavaScript зазвичай використовується як вбудована мова для програмного
      доступу до об'єктів додатків. Найбільш широке застосування знаходить у
      браузерах як мова сценаріїв для надання інтерактивності веб-сторінкам.
    </p>
  </div>
  <div>
    <button class='play' tabindex='3'></button>
    <p>
      Основні архітектурні риси: динамічна типізація, слабка типізація,
      автоматичне керування пам'яттю, прототипне програмування, функції
      як об'єкти першого класу.
    </p>
  </div>
</div>

```

У нас є три блоки (`div`) з кнопками для озвучування тексту (`play`) і, власне, текстом, який буде озвучуватися (перші три абзаци статті про `JavaScript` з Вікіпедії). *Зверніть увагу*, що я додав кнопкам атрибут `tabindex`, щоб перемикатися між ними за допомогою `tab` і натискати за допомогою `space`. Однак, майте на увазі, що використовувати атрибут `tabindex` не рекомендується через те, що браузер використовує перемикування фокусу за допомогою `tab` для покращення доступності.

З вашого дозволу, я наведу код скрипта повністю:

```

// Ця частина має бути вам знайома з попереднього прикладу
let voices = speechSynthesis.getVoices()
let defaultVoice

speechSynthesis.onvoiceschanged = () => {
  voices = speechSynthesis.getVoices()
  defaultVoice = voices.find((voice) => voice.name === 'Google український')

  wrapper.addEventListener('click', handleClick)
  window.addEventListener('keydown', handleKeydown)
}

```

```

const PLAY = 'play'
const PAUSE = 'pause'
const RESUME = 'resume'

function handleClick({ target }) {
  switch (target.className) {
    case PLAY:
      // При натисканні кнопки `play` в момент озвучування іншого тексту,
      // нам потрібно припинити поточне озвучування перед початком нового
      speechSynthesis.cancel()

      const { textContent } = target.nextElementSibling

      // Про цю частину див. нижче
      textContent.split('.').forEach((text) => {
        const trimmed = text.trim()
        if (trimmed) {
          const U = getUtterance(target, text)
          speechSynthesis.speak(U)
        }
      })
      break
    case PAUSE:
      // CSS-клас кнопки відповідає за іконку, що відображається поруч з нею
      // `⏸` - очікування/стоп, `⏹` - озвучування/відтворення, `⏸` - пауза
      // іконка `⏹` використовується як індикатор кнопки, що знаходиться у
      фокусі
      target.className = RESUME
      speechSynthesis.pause()
      break
    case RESUME:
      target.className = PAUSE
      speechSynthesis.resume()
      break
    default:
      break
  }
}

// При натисканні `escape` припиняємо озвучування тексту
// Можете додати свої "контроли"
function handleKeydown({ code }) {
  switch (code) {
    case 'Escape':
      return speechSynthesis.cancel()
    default:
      break
  }
}

function getUtterance(target, text) {
  const U = new SpeechSynthesisUtterance(text)
  U.voice = defaultVoice
}

```

```

U.lang = defaultVoice.lang
U.volume = 1
U.rate = 1
U.pitch = 1

// Я спеціально не став приховувати зміну іконок при початку/закінченні
відтворення
U.onstart = () => {
  console.log('Started')
  target.className = PAUSE
}
U.onend = () => {
  console.log('Finished')
  target.className = PLAY
}
U.onerror = (err) => console.error(err)

return U
}

```

Тонкий момент в приведенном коде - это преобразование озвучиваемого текста в массив предложений (разделителем является точка (.)), перебор массива, и воспроизведение каждого предложения по отдельности (точнее, помещение всех предложений одного за другим в очередь на озвучивание) - `textContent.split('.').forEach(...)`. Причина такого решения заключается в том, что озвучивание длинного текста тихо обрывается примерно на 220 символе (в **Chrome**). Черновик спецификации для такого случае предусматривает специальную ошибку **text-to-long** (текст является слишком длинным), но данной ошибки не возникает, озвучивание просто резко прекращается, причем, для восстановления работы **SpeechSynthesis** зачастую приходится перезагружать вкладку (при запущенном сервере для разработки даже это не всегда срабатывает). Возможно, вам удастся найти другое решение.

Поиграть с кодом можно [здесь](#).

Приложение для распознавания речи (перевода речи в текст)

После того, как мы обстоятельно рассмотрели **SpeechSynthesis**, можно переходить ко второму, немного более сложному, но, вместе с тем, и более интересному интерфейсу, входящему в состав **WSA - SpeechRecognition**.

Вот наша начальная разметка:

```

<div id="wrapper">
  <h1>Speech Recognition - Dictaphone</h1>
  <textarea id="final_text" cols="30" rows="10"></textarea>
  <input type="text" id="interim_text" />
  <div id="buttons">
    <button class="start">Старт</button>
    <button class="stop">Стоп</button>
    <button class="abort">Сброс</button>
  </div>

```



```
<button class="copy">Копия</button>
<button class="clear">Очистить</button>
</div>
</div>
```

У нас имеется поле для вставки финального (распознанного) текста (`final_text`) и поле для вставки промежуточного (находящегося в процессе распознавания) текста (`interim_text`), а также панель управления (`buttons`). Выбор элементов `textarea` и `input` для хранения текста обусловлен как вариативностью промежуточных результатов, которые меняются на лету, так и необходимостью внесения небольших правок в распознанный текст, что связано с естественным несовершенством перевода устной речи в текст. Стоит отметить, что, в целом, `Chrome` очень неплохо справляется с задачей распознавания речи и автоматическим форматированием распознанного текста.

Кроме кнопок для управления распознаванием речи (`start`, `stop` и `abort`), мы реализуем возможность копирования распознанного текста в буфер обмена с помощью `Clipboard API` и очистки соответствующего поля.

Начнем с определения переменных для финального текста и индикатора распознавания:

```
let final_transcript = ''
let recognizing = false
```

Далее, создаем и настраиваем экземпляр `SpeechRecognition`:

```
// Напоминаю, что `WSA`, в целом, и, особенно, `SpeechRecognition` являются
экспериментальными
const speechRecognition =
  window.SpeechRecognition || window.webkitSpeechRecognition
// Создаем экземпляр `SpeechRecognition`
const recognition = new speechRecognition()
// Свойство `continuous` определяет, продолжается ли распознавание речи после
получения первого финального результата
recognition.continuous = true
// обработка промежуточных результатов
recognition.interimResults = true
// максимальное количество альтернатив распознанного слова
recognition.maxAlternatives = 3
// язык
recognition.lang = 'ru-RU'
```

Добавляем обработчики событий запуска, ошибки и окончания распознавания речи:

```
recognition.onstart = () => {
  console.log('Распознавание голоса запущено')
}
```

```

recognition.onerror = ({ error }) => {
  console.error(error)
}
recognition.onend = () => {
  console.log('Распознавание голоса закончено')
  // Снова запускаем распознавание, если индикатор имеет значение `true`
  if (!recognizing) return
  recognition.start()
}

```

`SpeechRecognition` хорошо справляется с распознаванием слов, фраз и целых предложений, он даже выполняет вполне уместную "капитализацию" строки. Но он не "понимает" знаков препинания. Для того, чтобы помочь ему в этом определим словарь в виде объекта:

```

const DICTIONARY = {
  точка: '.',
  запятая: ',',
  вопрос: '?',
  восклицание: '!',
  двоеточие: ':',
  тире: '-',
  абзац: '\n',
  отступ: '\t'
}

```

Предполагается, что для решение задач, связанных с форматированием, будут использоваться специальные объекты `SpeechGrammar` и `SpeechGrammarList`, а также специальный синтаксис `JSpeech Grammar Format`, однако, в данной статье мы ограничимся обычным объектом.

Вы можете добавить в словарь любые пары ключ/значение, которые посчитаете нужными. Обратите внимание на то, что все ключи в нашем словаре представлены одним словом. Дело в том, что рассматриваемый интерфейс плохо справляется с ключами, которые состоят более чем из одного слова, например, "вопросительный знак", "восклицательный знак" и т.п. Я понимаю, что пара `вопрос: '?'` не лучшее решение, но для примера сойдет.

Также нам потребуются две функции для редактирования промежуточного и финального результатов:

```

function editInterim(s) {
  return s
    .split(' ')
    .map((word) => {
      word = word.trim()
      return DICTIONARY[word.toLowerCase()]
        ? DICTIONARY[word.toLowerCase()]
        : word
    })
}

```

```

        .join(' ')
    }

    function editFinal(s) {
        return s.replace(/\s{1,}([\.,?!:-])/g, '$1')
    }

```

Функция `editInterim()` разбивает фразу на массив слов, перебирает слова, удаляет пробелы в начале и конце слова и заменяет символом из словаря при совпадении. Обратите внимание, что мы приводим слово в нижний регистр только при поиске совпадения. Если мы сделаем это в строке `word = word.trim()`, то нивелируем автоматическую "капитализацию" строки, выполняемую браузером.

Функция `editFinal()` удаляет пробел перед каждым из указанных символов - мы разделяем слова пробелами при объединении в функции `editInterim()`. Следует отметить, что по умолчанию каждое распознанное слово с двух сторон обрамляется пробелами.

Итак, событие, которое интересует нас больше всего - это `result`. Реализуем его обработку:

```

recognition.onresult = (e) => {
    // Промежуточные результаты обновляются на каждом цикле распознавания
    let interim_transcript = ''
    // Перебираем результаты с того места, на котором остановились в прошлый раз
    for (let i = e.resultIndex; i < e.results.length; i++) {
        // Атрибут `isFinal` является индикатором того, что речь закончилась
        if (e.results[i].isFinal) {
            // Редактируем промежуточные результаты
            const result = editInterim(e.results[i][0].transcript)
            // и добавляем их к финальному
            final_transcript += result
        } else {
            // В противном случае, записываем распознанные слова в промежуточный
результат
            interim_transcript += e.results[i][0].transcript
        }
    }
    // Записываем промежуточные результаты в `input`
    interim_text.value = interim_transcript
    // Редактируем финальный результат
    final_transcript = editFinal(final_transcript)
    // и записываем его в `textarea`
    final_text.value = final_transcript
}

```

Событие распознавания выглядит следующим образом:

```

SpeechRecognitionEvent
  bubbles: false

```

```

cancelBubble: false
cancelable: false
composed: false
currentTarget: SpeechRecognition {grammars: SpeechGrammarList, lang: "ru-RU",
continuous: true, interimResults: true, maxAlternatives: 3, ...}
defaultPrevented: false
emma: null
eventPhase: 0
interpretation: null
isTrusted: true
path: []
resultIndex: 1
// здесь нас интересуют только результаты
results: SpeechRecognitionResultList {0: SpeechRecognitionResult, 1:
SpeechRecognitionResult, length: 2}
returnValue: true
srcElement: SpeechRecognition {grammars: SpeechGrammarList, lang: "ru-RU",
continuous: true, interimResults: true, maxAlternatives: 3, ...}
target: SpeechRecognition {grammars: SpeechGrammarList, lang: "ru-RU",
continuous: true, interimResults: true, maxAlternatives: 3, ...}
timeStamp: 59862.61999979615
type: "result"

```

Результаты (`SpeechRecognitionResultList`) выглядят так:

```

results: SpeechRecognitionResultList
  0: SpeechRecognitionResult
    0: SpeechRecognitionAlternative
      confidence: 0.7990190982818604
      transcript: "привет"
      isFinal: true
      length: 1
    length: 1
  length: 1

```

Вот почему для получения результата мы обращаемся к `e.results[i][0].transcript`. На самом деле, поскольку мы указали `maxAlternatives = 3`, во всех результатах будет представлено по три `SpeechRecognitionAlternative`. Первым (с индексом 0) всегда будет наиболее подходящий результат с точки зрения браузера.

Все, что нам осталось сделать, это реализовать обработку нажатия кнопок:

```

buttons.onclick = ({ target }) => {
  switch (target.className) {
    case 'start':
      // Обнуляем переменную для финального результата
      final_transcript = ''
      // Запускаем распознавание
      recognition.start()
  }
}

```

```

    // Устанавливаем индикатор распознавания в значение `true`
    recognizing = true
    // Очищаем `textarea`
    final_text.value = ''
    // Очищаем `input`
    interim_text.value = ''
    break
case 'stop':
    // Останавливаем распознавание
    recognition.stop()
    // Устанавливаем значение индикатора распознавания в значение `false`
    recognizing = false
    break
case 'abort':
    // Прекращаем распознавание
    recognition.abort()
    recognizing = false
    break
case 'copy':
    // Копируем текст из `textarea` в буфер обмена
    navigator.clipboard.writeText(final_text.value)
    // Сообщаем об этом пользователю
    target.textContent = 'Готово'
    const timerId = setTimeout(() => {
        target.textContent = 'Копия'
        clearTimeout(timerId)
    }, 3000)
    break
case 'clear':
    // Обнуляем переменную для финального результата
    final_transcript = ''
    // Очищаем `textarea`
    final_text.value = ''
    break
default:
    break
}
}

```

Тестируем. Нажимаем на кнопку "Старт", ждем, когда красная точка рядом с фавиконкой перестанет мигать (о готовности к распознаванию можно сообщать пользователю через обработку события `speechstart`), произносим фразу (произношение должно быть максимально четким и ясным), например, "привет точка как дела вопрос". В `input` на какое-то время появляется "Привет точка Как дела вопрос", затем этот промежуточный результат редактируется и переносится в `textarea` в виде "Привет. Как дела?". Отлично, наша машина нас понимает.

Поиграть с кодом можно [здесь](#).

Подумал о том, что было бы неплохо реализовать функцию для удаления последнего распознанного слова на тот случай, если результат распознавания получился некорректным. Для этого потребуется пара `удалить: () => removeLastWord()` (если добавить эту пару в словарь, то

потребуется дополнительная проверка `typeof DICTIONARY[word] === 'function'`) и примерно такая операция:

```
function removeLastWord() {
  const oldStr = final_text.value
  const newStr = oldStr.substring(0, oldStr.lastIndexOf(' '))
  final_text.value = newStr
}
```

Однако, добавлять операцию в словарь - плохая идея, для этого лучше использовать отдельный объект, чем мы и займемся в следующем разделе.

Одностраничное приложение с возможностью голосового управления

Такой вариант использования `SpeechRecognition`, на мой взгляд, представляет наибольший интерес с точки зрения возможности применения данной технологии в реальных приложениях.

Мы будем генерировать страницы программным способом, поэтому разметка нам не потребуется.

Создаем директорию `pages` с тремя файлами:

```
// home.js
export default /*html*/ `
<div id="wrapper">
  <div>Section 1</div>
  <div>Section 2</div>
  <div>Section 3</div>
  <div>Section 4</div>
  <div>Section 5</div>
  <div>Section 6</div>
  <div>Section 7</div>
  <div>Section 8</div>
  <div>Section 9</div>
</div>
`

// product.js
export default /*html*/ `
<h1>This is the Product Page</h1>
`

// about.js
export default /*html*/ `
<h1>This is the About Page</h1>
`
```

Основной скрипт начинается с импорта страниц и генерации домашней страницы:

```
import HomePage from './pages/home.js'
import ProductPage from './pages/product.js'
import AboutPage from './pages/about.js'

const { body } = document
body.innerHTML = HomePage
```

Нам потребуются константы для прокрутки страницы и объект с операциями:

```
// Константы для прокрутки
const DOWN = 'down'
const UP = 'up'
const RIGHT = 'right'
const LEFT = 'left'

// Операции
const ACTIONS = {
  // операции переключения страниц
  home: () => (body.innerHTML = HomePage),
  product: () => (body.innerHTML = ProductPage),
  about: () => (body.innerHTML = AboutPage),

  // операции прокрутки
  down: () => scroll(DOWN),
  up: () => scroll(UP),
  left: () => scroll(LEFT),
  right: () => scroll(RIGHT),

  // операции переключения цветовой темы
  light: () => body.removeAttribute('class'),
  dark: () => (body.className = 'dark')
}
```

Далее следуют настройки `SpeechRecognition` и обработчики событий начала, ошибки и окончания распознавания, аналогичные рассмотренным в предыдущем разделе (кроме настройки языка - для управления страницей мы будем использовать американский английский: `recognition.lang = 'en-US'`).

Обработка события `result`:

```
recognition.onresult = (e) => {
  // Перебираем результаты
  for (let i = e.resultIndex; i < e.results.length; i++) {
    if (e.results[i].isFinal) {
      const result = e.results[i][0].transcript.toLowerCase()
      // Выводим слова в консоль, чтобы убедиться в корректности распознавания
      (как выяснилось, слово `product` очень плохо распознается, возможно, у меня
```

```

проблемы с его правильным произношением)
    console.log(result)
    // Преобразуем фразу в массив, перебираем слова и выполняем
    соответствующие операции
    result.split(' ').forEach((word) => {
        word = word.trim().toLowerCase()
        // ACTION[word] - это функция
        return ACTIONS[word] ? ACTIONS[word]() : ''
    })
}
}
}

```

Функция для выполнения прокрутки выглядит следующим образом:

```

function scroll(direction) {
    let newPosition
    switch (direction) {
        case DOWN:
            newPosition = scrollY + innerHeight
            break
        case UP:
            newPosition = scrollY - innerHeight
            break
        case RIGHT:
            newPosition = scrollX + innerWidth
            break
        case LEFT:
            newPosition = scrollX - innerWidth
            break
        default:
            break
    }
    if (direction === DOWN || direction === UP) {
        scrollTo({
            top: newPosition,
            behavior: 'smooth'
        })
    } else {
        scrollTo({
            left: newPosition,
            behavior: 'smooth'
        })
    }
}
}

```

Наконец, добавляем обработку нажатия клавиш клавиатуры:


```
window.addEventListener('keydown', (e) => {
  e.preventDefault()
  switch (e.code) {
    // Нажатие пробела запускает распознавание
    case 'Space':
      recognition.start()
      recognizing = true
      break
    // Нажатие `escape` останавливает распознавание
    case 'Escape':
      recognition.stop()
      recognizing = false
    default:
      break
  }
})
```

Проверяем работоспособность приложения. Нажимаем пробел, ждем готовности браузера к распознаванию речи, произносим следующие фразы:

- `home`, `product`, `about` - переключение страниц
- `dark`, `light` - переключение цветовой темы
- `down`, `up`, `left`, `right` - выполнение прокрутки к соответствующему разделу страницы (имеет видимый эффект только на домашней странице; иногда приходится добавлять слово `scroll`, например, `scroll down`)

Поиграть с кодом можно [здесь](#).